

過学習と汎化

モデルの「本当の実力」を測る

rkskmt

1

前回の振り返り

2次元分類と「あやしい満点」

- 重さ + 輝度の **2特徴量** で Salmon / Sea bass を分類
- 決定木・ロジスティック回帰で決定境界を学習した
- **深さ無制限の決定木は正答率 100%** — ただし採点は**学習に使った同じデータ**
- 境界には、数匹のためだけの**飛び地**ができていた

📌 今回の問い

あの満点は **本当に信頼できる** のか？

2

何が問題か

- モデルは、学習データの特徴を見てパラメータを調整する
- その同じデータで採点すると、精度は**高めにしやすい**
- 知りたいのは「覚えたデータへの強さ」ではない
- 知りたいのは「次に来る魚にも通用するか」

⚠️ 学習データで採点する危険

- 学習に使ったデータでの精度は、モデルの実力を楽観的に見積もる可能性がある
- まずは、前回と同じ採点方法をもう一度確認する

3

汎化という考え方

- 勉強には2種類のうまく見える状態がある

方法	内容	本当の実力？
暗記	去年の問題を全部覚える	✗ 今年の問題は解けない
理解	解き方を身に付ける	✓ 未知の問題にも対応できる

- 機械学習モデルでも、同じ区別が必要

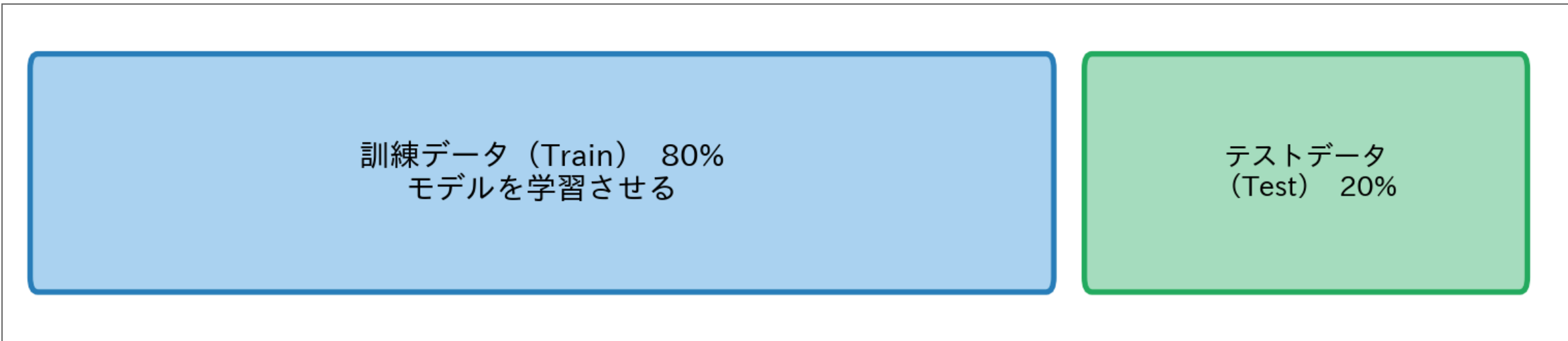
📌 汎化 (generalization)

- モデルが**学習していないデータ**にも通用する能力のこと

4

訓練データとテストデータ

- データを2つに分けて使う



- **訓練データ**：モデルに見せて学習させる
- **テストデータ**：学習後に **一度だけ** 使い「本当の精度」を測る

⚠ テストデータの鉄則

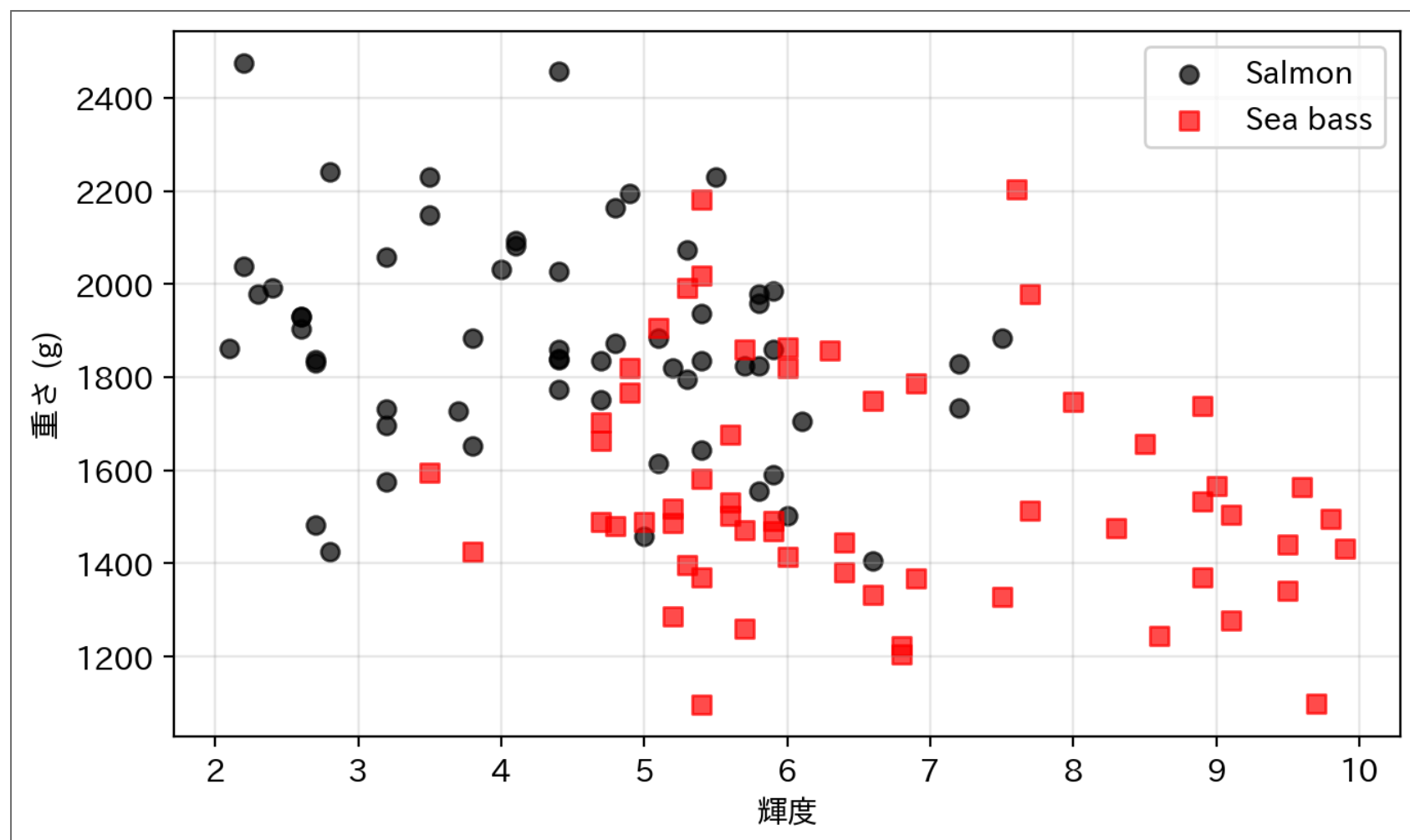
- テストデータを確認してからモデルを調整しない
- 「見たことのない問題」を模擬するために残しておく

データの準備 — 魚を増やす

- 前回の80匹に、新しく測った **40匹** を加えた **120匹** で考える
- データが増えると、**どちらとも言いきくい微妙な個体** や計測のばらつきも入ってくる

▶ 魚データの定義（クリックして展開）

まずデータを眺める



- 「左上が Salmon・右下が Sea bass」の傾向は前回と同じ
- ただし真ん中に、2種が**入り混じるゾーン**ができている
- どんな境界線を引いても、このゾーンの**多少の間違いは避けられない**

⚠ ここで予想

このデータで「正答率 100%」を出すモデルがあったら — それは天才か、それとも？

7

前回と同じ採点方法

- まずは全データで学習し、同じ全データで評価する
- これは「学習済みデータへの精度」を測っている

Code

```
1 from sklearn.linear_model import LogisticRegression
2 from sklearn.metrics import accuracy_score
3
4 same_data_model = LogisticRegression(max_iter=1000)
5 same_data_model.fit(X, y)
6
7 same_data_acc = accuracy_score(y, same_data_model.predict(X))
8 print(f"全データで学習 → 同じ全データで評価: {same_data_acc:.3f}")
```

Result

全データで学習 → 同じ全データで評価: 0.800

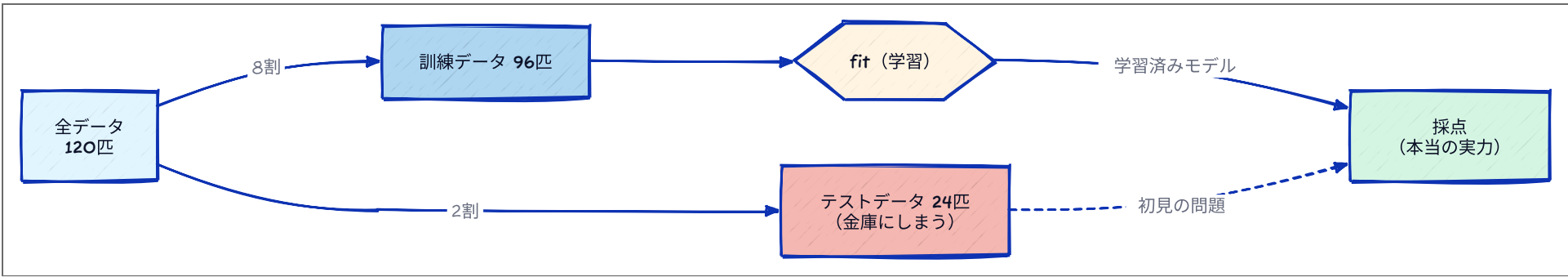
📌 ノート

- この値だけでは、新しい魚にも通用するかは分からない
- 次に、データを「学習用」と「評価用」に分ける

8

ホールドアウト法

- データの一部を、最初から評価用に取り分ける
- 手軽なので、最初に試す評価方法としてよく使う



Code

```
1 from sklearn.model_selection import train_test_split
2
3 # 8:2 に分割 (test_size=0.2 で20%をテストに)
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y, test_size=0.2, random_state=42
6 )
7
8 print(f"全データ:      {len(X)} 匹")
9 print(f"訓練データ:    {len(X_train)} 匹")
10 print(f"テストデータ: {len(X_test)} 匹")
```

Result

```
全データ:      120 匹
訓練データ:    96 匹
テストデータ:  24 匹
```

💡 random_state とは

- 分割はランダムなので毎回変わる
- **random_state=42** のように数値を固定すると再現できる

見ていないデータで評価する

Code

```
1 model = LogisticRegression(max_iter=1000)
2 model.fit(X_train, y_train)          # 訓練データだけで学習
3
4 train_acc = accuracy_score(y_train, model.predict(X_train))
5 test_acc  = accuracy_score(y_test,  model.predict(X_test))
6
7 print(f"訓練データ精度: {train_acc:.3f} ← 学習済みデータへの精度 (楽観的) ")
8 print(f"テストデータ精度: {test_acc:.3f} ← 本当の精度")
```

Result

訓練データ精度: 0.802 ← 学習済みデータへの精度 (楽観的)
テストデータ精度: 0.792 ← 本当の精度

💡 2つの精度を比べる

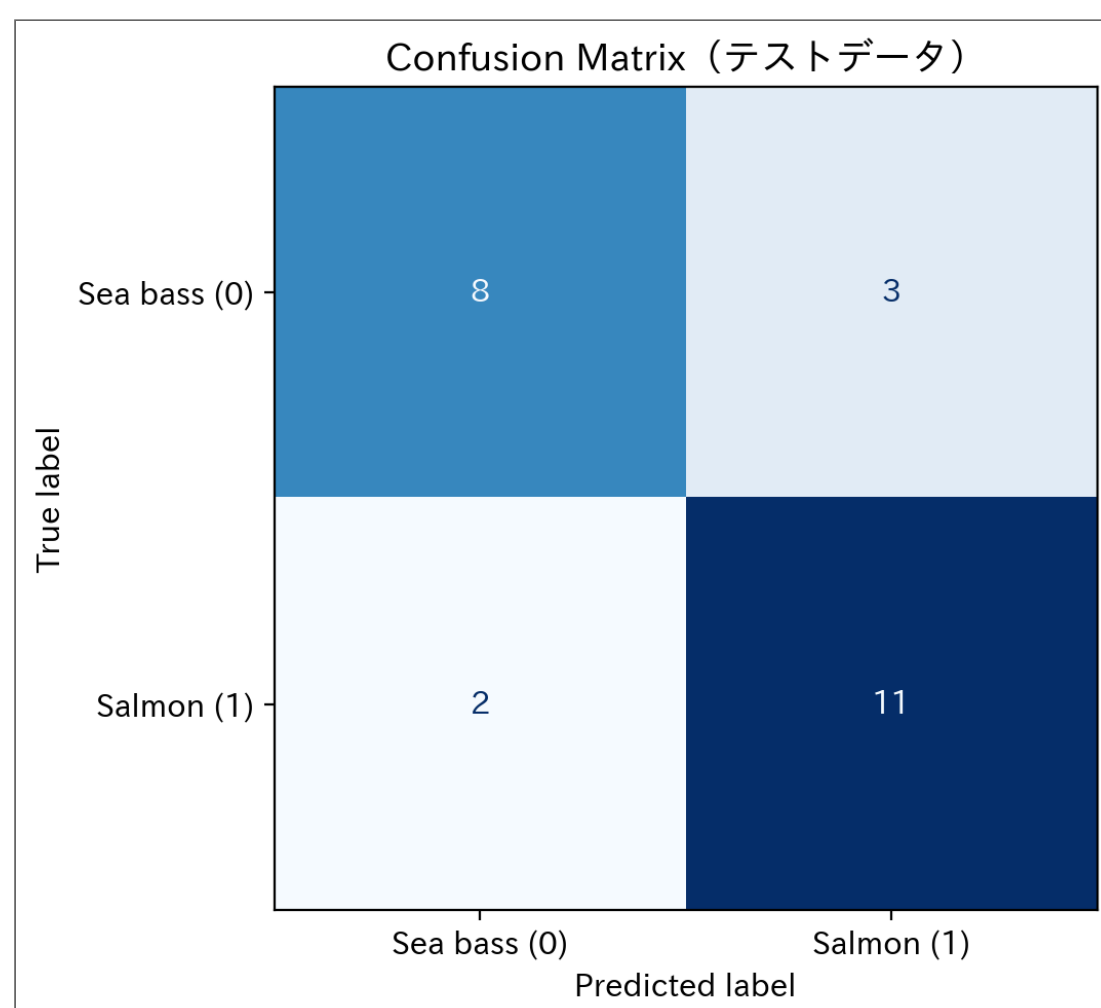
- 訓練精度 ≈ テスト精度：うまく汎化できている
- 訓練精度 >> テスト精度：過学習（**overfitting**）の疑い（暗記している）

10

混同行列（テストデータで評価）

Code

```
1 from sklearn.metrics import ConfusionMatrixDisplay, confusion_matrix
2
3 cm = confusion_matrix(y_test, model.predict(X_test))
4 disp = ConfusionMatrixDisplay(confusion_matrix=cm,
5                               display_labels=["Sea bass (0)", "Salmon (1)"])
6 disp.plot(colorbar=False, cmap="Blues")
7 plt.title("Confusion Matrix (テストデータ) ")
8 plt.tight_layout()
9 plt.show()
```



11

深さ無制限の決定木を、新しい魚で採点する

- 前回の「あやしい満点」に、いま手に入れた道具（train/test）を使う

❗ ここで予想

訓練で満点だったこの木 — 新しい24匹のうち、何匹当てられる？

```
Code
1 from sklearn.tree import DecisionTreeClassifier
2
3 deep_tree = DecisionTreeClassifier(max_depth=None, random_state=42)
4 deep_tree.fit(X_train, y_train)           # 訓練データだけで学習
5
6 deep_train_acc = accuracy_score(y_train, deep_tree.predict(X_train))
7 deep_test_acc  = accuracy_score(y_test,  deep_tree.predict(X_test))
8
9 print(f"訓練データ精度: {deep_train_acc:.3f}")
10 print(f"テストデータ精度: {deep_test_acc:.3f}")
```

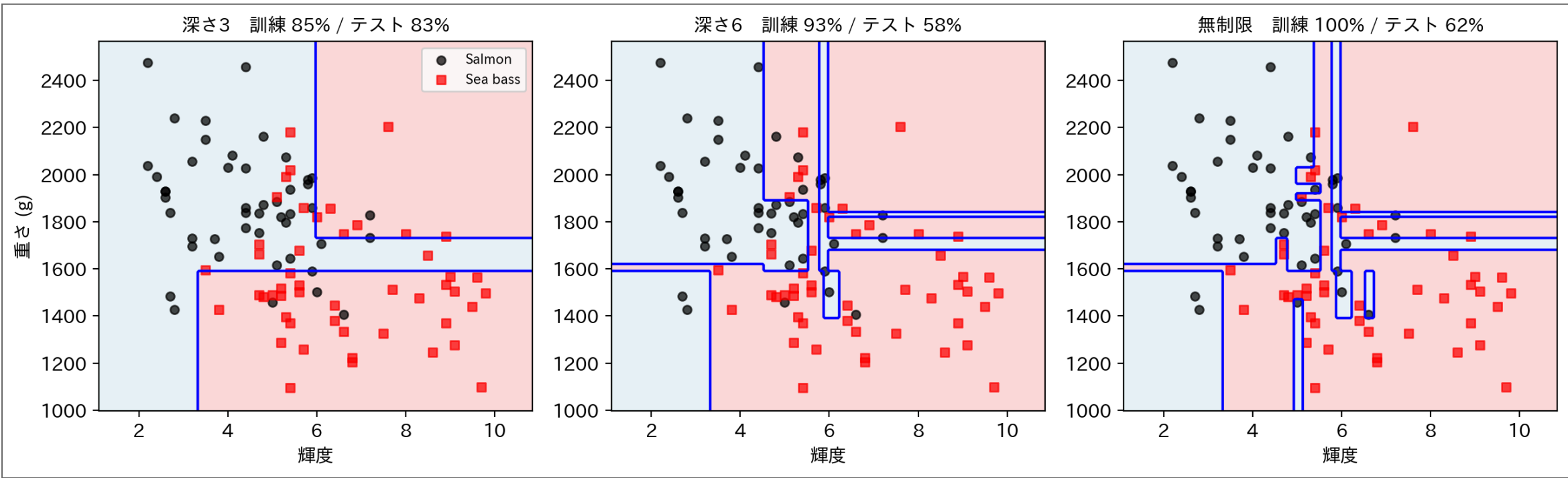
Result

訓練データ精度: 1.000
テストデータ精度: 0.625

- 訓練では**満点**。ところが新しい魚では**4割近く間違える**
- 満点の正体は実力ではなく**暗記**だった

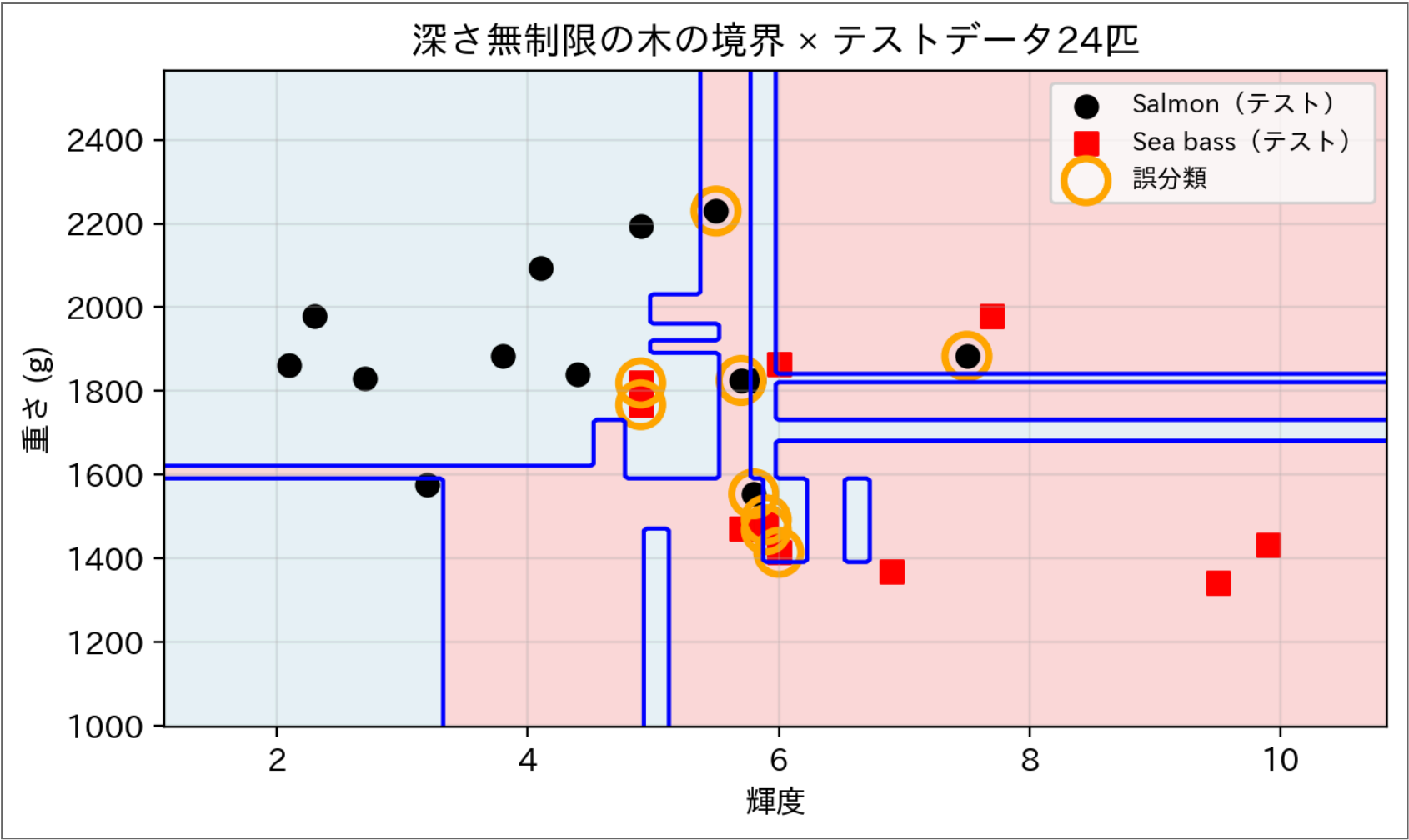
境界線はどう「学習」されていたのか

- 深さを変えた木の**決定境界**を、訓練データと一緒に描いてみる



- 分割を増やすほど、入り混じりゾーンに**細い回廊や飛び地**が増えていく
- 無制限の木は、微妙な1匹1匹を救うために**境界をめちゃくちゃに曲げて**訓練データ満点を達成している

暗記の境界に、新しい魚を流し込む

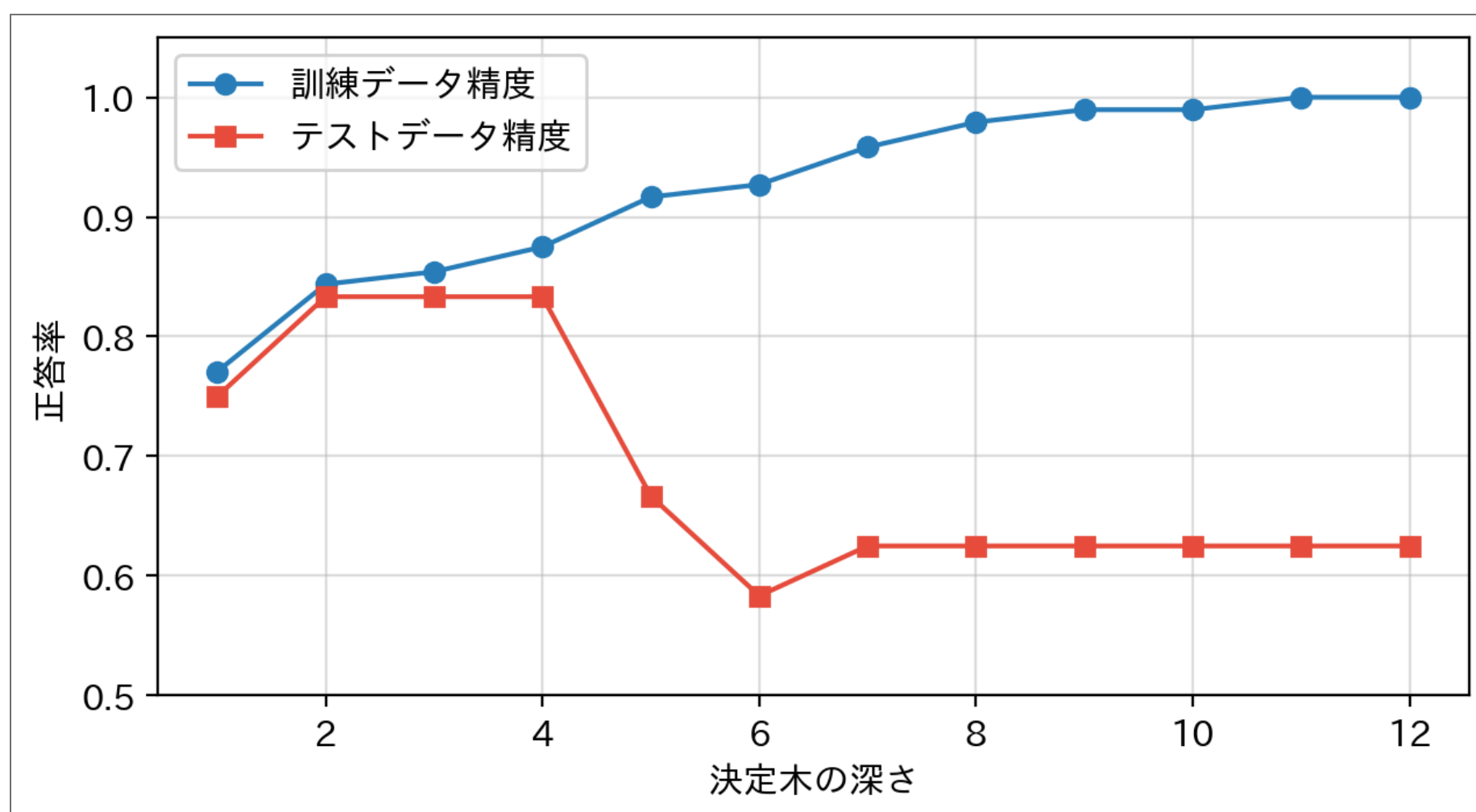


- 回廊や飛び地は、**訓練データの特定の数匹**のためのオーダーメイド
- 新しい魚はその意図どおりには並ばない — 誤分類（オレンジ）は入り混じりゾーンに集中
- 暗記した細部は、**次に来る魚には通用しない**

深さを変えて、2つの精度を並べる

Code

```
1 depths = list(range(1, 13))
2 train_accs, test_accs = [], []
3
4 for d in depths:
5     m = DecisionTreeClassifier(max_depth=d, random_state=42).fit(X_train, y_train)
6     train_accs.append(accuracy_score(y_train, m.predict(X_train)))
7     test_accs.append(accuracy_score(y_test, m.predict(X_test)))
8
9 plt.figure(figsize=(6.5, 3.6))
10 plt.plot(depths, train_accs, "o-", color="#2980b9", label="訓練データ精度")
11 plt.plot(depths, test_accs, "s-", color="#e74c3c", label="テストデータ精度")
12 plt.xlabel("決定木の深さ"); plt.ylabel("正答率")
13 plt.ylim(0.5, 1.05)
14 plt.legend(); plt.grid(True, alpha=0.4)
15 plt.tight_layout()
16 plt.show()
```



- 訓練精度は深くするほど上がり続け、やがて 100% に到達する
- テスト精度は **深さ2~4がピーク**。それ以上深くすると **むしろ下がる**
- 深さ5以降、訓練精度だけが伸びて2本の線の **差（ギャップ）** が広がる — 暗記が進んでいるサイン

過学習（Overfitting）

- 訓練データの細部（たまたまの並び・ノイズ）まで覚え込み、**新しいデータで性能が落ちること**
- 数字での証拠は**訓練精度とテスト精度のギャップ**
- 図での証拠は**数匹のためだけの回廊・飛び地**だらけの決定境界
- モデルが複雑なほど（深い木・多すぎるパラメータ）起きやすい

状態	訓練精度	テスト精度
未学習（underfitting）	低い	低い
ちょうど良い	高い	高い
過学習（overfitting）	ほぼ満点	低い

💡 対策

- モデルの複雑さを抑える（決定木なら**深さを制限**）
- データを増やす
- テスト精度を安定して測る — それが次の**交差検証（cross-validation）**

17

交差検証

18

ホールドアウト法の問題点

- **train_test_split** は**ランダムに分割**する
- どのデータがテスト側に入るかで、精度が変わる

```
Code
1 for seed in range(5):
2     Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=seed)
3     acc = accuracy_score(yte, LogisticRegression(max_iter=1000).fit(Xtr, ytr).predict(Xte))
4     print(f"seed={seed}   テスト精度: {acc:.3f}")
```

```
Result
|
| seed=0   テスト精度: 0.833
| seed=1   テスト精度: 0.917
| seed=2   テスト精度: 0.792
| seed=3   テスト精度: 0.833
| seed=4   テスト精度: 0.917
|
```

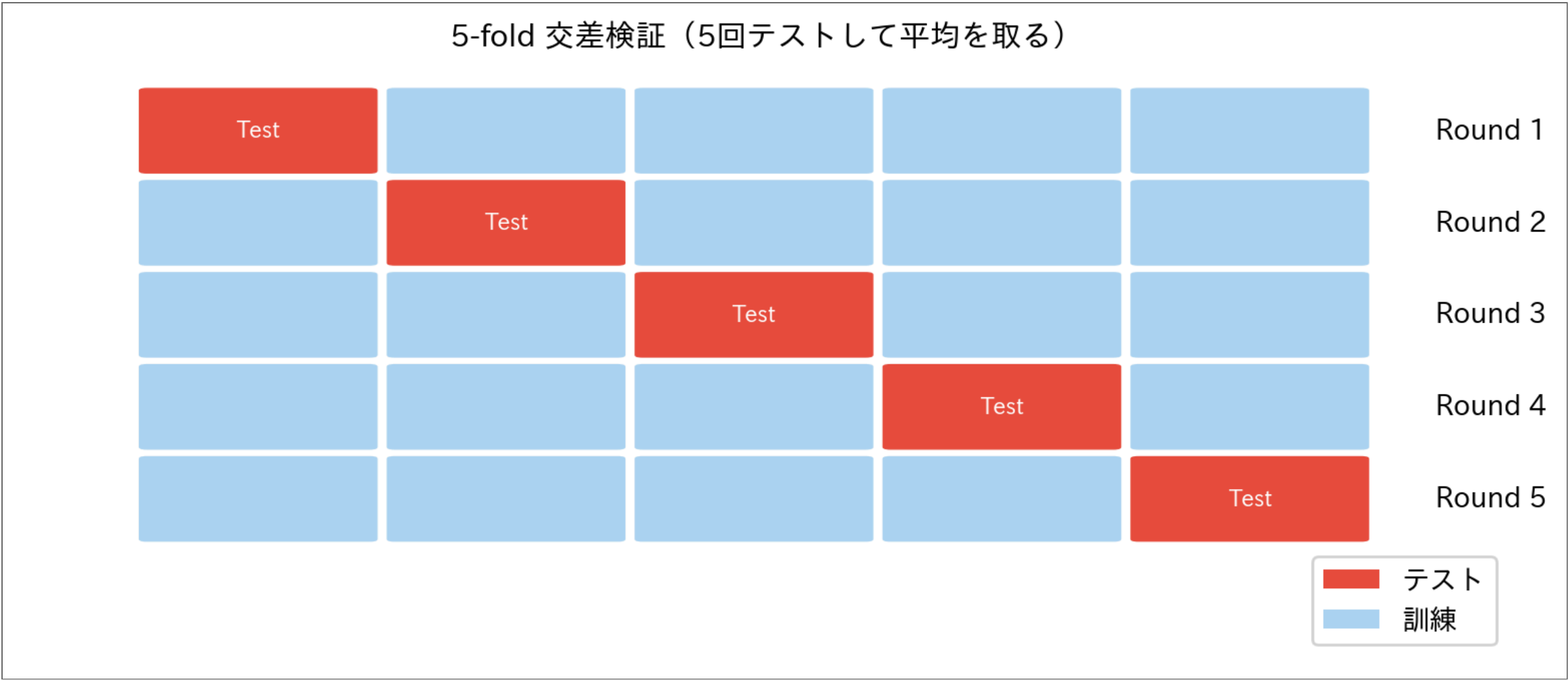
⚠ データが少ないほど揺れが大きい

- テストデータが24匹の場合、1匹の誤分類で精度が約 4% 変わる
- 1回だけの分割結果を、強く信じすぎない

19

k-fold 交差検証

- データを **k 個のグループ (fold)** に分割する
- k 回繰り返す：1つをテスト、残りを訓練に使う
- k 個の精度の **平均** を最終スコアとする



20

cross_val_score で交差検証

```
Code
1 from sklearn.model_selection import cross_val_score
2
3 scores = cross_val_score(
4     LogisticRegression(max_iter=1000), X, y, cv=5 # cv=5 で 5-fold
5 )
6
7 print("各 fold の精度:", scores.round(3))
8 print(f"平均精度: {scores.mean():.3f} ± {scores.std():.3f}")
```

Result

各 fold の精度: [0.792 0.833 0.792 0.75 0.792]

平均精度: 0.792 ± 0.026

💡 cross_val_score の引数

- 第1引数：モデル (**fit** 前でOK。内部で k 回 fit してくれる)
- cv=k**：k-fold の k を指定 (よく使うのは 5 か 10)
- 返り値は k 個の精度の配列 → **.mean()** / **.std()** で集計

21



- ホールドアウトのコード と 交差検証のコード を書き換えて実行（精度のセルまで続けて実行）

1	分け方の運を変える	<code>random_state=42</code> → <code>1</code>
2	テストに半分回す	<code>test_size=0.2</code> → <code>0.5</code>
3	10-fold にする	<code>cv=5</code> → <code>cv=10</code>

💡 ここで観察

- `random_state=1` だとテスト精度 **91.7%**（42 では 79.2%） — **同じモデルなのに12ポイント上振れ**。1回の分割の数字には運が乗っている — だから交差検証
- `test_size=0.5` は訓練が60匹に減り、テスト精度 78.3% — 訓練とテストの**取り分のトレードオフ**
- `cv=10` は平均 0.80 とほぼ同じだが、ばらつきは ± 0.026 → **± 0.085** が増える — 1 fold が12匹しかないため。fold は細かくすればいいものでもない

22

まとめ

- 訓練データで評価した精度は**楽観的** — 満点はむしろ疑う
- **過学習**：訓練データを暗記して、新しいデータで性能が落ちること
 - 証拠は**訓練精度とテスト精度のギャップ**（深さ無制限の決定木：100% vs 63%）
 - 決定境界で見ると、数匹のためだけの**回廊・飛び地**として現れる
- **汎化**：未知データへの対応能力を測ることが本来の目的
- `train_test_split`：手軽だが分割次第で精度が揺れる
- **k-fold 交差検証**：k 回の平均で安定した評価が得られる

手法	特徴
ホールドアウト	手軽。データが多いときに有効
k-fold 交差検証	安定。データが少ないときに特に有効

📄 次回予告

「分類にはラベルが必要だった。**ラベルがない場合はどうする？**」

23

練習問題

問題1は手計算、問題2はPythonコードで解け。

問題1：どちらのモデルを仕入れる？

新しい魚20匹（Salmon 10・Sea bass 10）で、モデルAを試したら：Salmon は10匹中8匹を正しく Salmon と判定、Sea bass は10匹中9匹を正しく Sea bass と判定した。

- 1. モデルAの混同行列を書け
- 2. モデルAの正答率を求めよ
- 3. モデルBは訓練データで100%、この20匹では70%だった。工場に納品するならどちらか、理由とともに答えよ

問題2：ちょうど良い深さを交差検証で選ぶ

本編の魚データ X, y を使って：

- 1. 深さ 1～10 の決定木それぞれについて、5-fold 交差検証の平均精度を求めよ
- 2. 平均精度が最大になる深さを選べ（訓練精度は見ない）

問題1の解答例

混同行列（行＝正解、列＝予測）：

	Salmon と予測	Sea bass と予測
正解 Salmon	8	2
正解 Sea bass	1	9

正答率: $(8 + 9)/20 = 85\%$

選ぶのはモデルA。モデルBの「訓練100%・新データ70%」は典型的な過学習のギャップ。性能は見たことのないデータへの正答率（汎化性能）で比べる — 訓練の成績は当てにならない。

問題2の解答例

▶ 解答例を見る

Result

深さ	1:	平均精度	0.633
深さ	2:	平均精度	0.658
深さ	3:	平均精度	0.683
深さ	4:	平均精度	0.683
深さ	5:	平均精度	0.650
深さ	6:	平均精度	0.625
深さ	7:	平均精度	0.608
深さ	8:	平均精度	0.608
深さ	9:	平均精度	0.617
深さ	10:	平均精度	0.608

★ 交差検証で選んだ深さ: 3

- 深くするほど良くなるわけではない — 平均精度は途中で**頭打ちか下り坂**になる
- 訓練精度で選んだら深さ10一択（そして過学習）。**複雑さは、見ていないデータに選ばせる**